

Comprehensive Client Specifications: Technology Stacks & End-to-End System Workflows

This document presents a complete, high-level and detailed architectural blueprint of the WAVEINIT Enterprise Learning Management System (LMS), AI-powered Assessment Generator, Containerized Code Execution Sandbox, Native WebRTC Live Interview Suite, and Automated AI Proctoring Engine. Prepared specifically for client review and implementation alignment.

Document Type:	Client Technical & Workflow Specification
Target Audience:	Enterprise Clients, Stakeholders, Technical Architects
System Core:	Enterprise LMS, AI Question Generator, Code Sandbox, Native WebRTC
Security & Compliance:	Role-Based Access Control (RBAC), Docker Isolation, In-House Signalling
Release Version:	v2.4 Production Standard (August 2026)
Document Status:	APPROVED FOR CLIENT DISTRIBUTION

EXECUTIVE NOTE FOR CLIENTS:

WAVEINIT is built from the ground up to eliminate third-party software dependencies. All live video/audio streams run over native in-house WebRTC with custom Socket.IO signalling (no Zoom or Teams required). Code execution is isolated via Docker containers, and assessment creation is automated through RAG-based AI microservices.

1. Executive Summary & System Architecture

The **WAVEINIT Enterprise LMS & Assessment Platform** is a unified, multi-tenant capable educational and talent evaluation ecosystem designed for enterprises, universities, and training organizations. It bridges corporate training, automated skill evaluation, real-time proctored exams, and technical interviewing into a single secure platform.

Core System Pillars

- **1. Course & Content Hub:** Hierarchical course creation, lesson management, rich multi-format assets (PDF, Video, Links, Slide Decks).
- **2. AI Question & Quiz Generator:** RAG-driven microservice parsing enterprise training materials to generate Bloom's taxonomy tagged quizzes instantly.
- **3. Containerized Code Execution:** Isolated, sandbox environment for running student code in Python, JavaScript, C++, and Java with automated test case validation.
- **4. Native WebRTC Interview Suite:** In-app peer-to-peer technical interview rooms featuring dual-camera mobile pairing, synchronized live coding editor, and chunked recording.
- **5. Automated AI Proctoring:** Multi-layered exam proctoring detecting tab switching, fullscreen exit, clipboard abuse, and candidate identity anomalies.

High-Level Architectural Topography

Layer	Components & Technologies	Primary Function
Presentation Layer	React 18, Vite, Tailwind CSS v4, Monaco Editor, Socket.io Client, Chart.js	Responsive Client UI, Interactive Code IDE, Live Video Call, Proctoring Telemetry
API & Application Server	Node.js, Express.js framework, Sequelize ORM, JWT Authentication, Winston Logging	Business Logic, RBAC Authorization, REST Endpoints, DB Transaction Management
Real-Time Signalling	Socket.IO Server, Redis Adapter, WebRTC RTCPeerConnection, STUN/TURN	WebRTC Signalling, Live Code Sync, Real-time Proctoring Alerts, Mobile QR Pairing
AI Microservice	Python FastAPI, LangChain, FAISS Vector Store, Google Gemini / OpenAI GPT-4	Document RAG parsing, Automatic Question Generation, AI Candidate Evaluation
Execution Sandbox	Docker Engine API (`dockerode`), Judge0 Microservice, Linux cgroups isolation	Safe execution of untrusted user code against test cases with time/memory caps
Data & Storage Layer	MySQL 8.0 Database, Redis Cache & BullMQ Queues, Cloudinary Media Storage	Persistent Relational Storage, Async Task Queues, Session Caching, Video Chunk Storage

2. Full Technology Stack Specifications

The platform utilizes a modern, decoupled microservices-friendly architecture designed for high scalability, fault tolerance, and security. Below is the comprehensive technology stack breakdown for each tier.

Frontend Technology Stack

Category	Technology / Library	Version / Specification	Purpose / Usage
Core Framework	React	v18.2.0 (Vite Bundler)	Component-driven single page application framework
Styling Engine	Tailwind CSS	v4.2.4 + Custom Tokens	Utility-first design system with custom theme variables
Code Editor IDE	Monaco Editor	@monaco-editor/react v4.6	VS Code style browser IDE for coding tests & interviews
Rich Text Editor	Tiptap	v2.27 Starter Kit	WYSIWYG editor for lesson notes, instructions, and posts
Real-Time Socket	Socket.IO Client	v4.7.2	Signalling, real-time code sync, and proctoring telemetry
Data Charts	Chart.js & Recharts	Chart.js v4.5 / Recharts v2.10	Visualizing participant metrics, score distributions & analytics
UI Animations	Framer Motion	v12.38.0	Smooth transitions, modal animations, micro-interactions
Icons & QR	Lucide React & QRCode	v1.14 / v4.2	Modern iconography and mobile device pairing QR generator

Backend Application Server Stack

Category	Technology / Library	Version / Specification	Purpose / Usage
Runtime Environment	Node.js	v18+ LTS	Event-driven JavaScript server runtime
Web Server Framework	Express.js	v4.18.2	REST API routing, middleware pipeline, controller handlers
Database ORM	Sequelize ORM	v6.35.2	Object-relational mapping for MySQL / PostgreSQL schema
Authentication & Crypt	JWT & BcryptJS	JWT v9.0 / Bcrypt v2.4	Stateful/Stateless token authentication and password hashing
Queue & Caching	BullMQ & Redis	BullMQ v5.79 / Redis v6.1	Background job queues for code grading, emails, reports
Real-Time Engine	Socket.IO + Redis Adapter	Socket.IO v4.7 / Adapter v8.1	Horizontally scalable WebSocket messaging server
Security Hardening	Helmet, Rate-Limit, HPP	Helmet v8.3 / Rate-Limit v8.5	HTTP security headers, DDoS rate limiting, parameter protection
Document Processing	PDF-Parse, Mammoth, ExcelJS	PDF v1.1 / Mammoth v1.12	Parsing PDF/Word uploaded materials & generating Excel reports

AI Microservice & RAG Engine Stack

Category	Technology / Library	Version / Specification	Purpose / Usage
Microservice Framework	FastAPI & Uvicorn	FastAPI v0.115 / Uvicorn v0.34	Asynchronous high-performance Python REST API
LLM Framework	LangChain & LangGraph	LangChain v1.3 / LangGraph v1.2	Chaining prompts, RAG document pipelines, state machines
AI Generative Models	Google Gemini & OpenAI	Gemini 1.5 Pro / GPT-4o	Automated question generation, code analysis, candidate feedback
Vector Store & RAG	FAISS & Sentence-Transformers	FAISS v1.8 / Transformers v3.0	In-memory similarity search for document embeddings
Doc Extractors	PyMuPDF, docx, pptx	PyMuPDF v1.24 / Docx v1.1	Extracting text content from PDF, DOCX, PPTX lesson materials

3. System Roles & Access Control Governance

The system enforces strict Role-Based Access Control (RBAC) across three primary authorization tiers. Every API request is checked against cryptographic JWT claims and database role state.

Capability / Action	System Admin	Trainer / Interviewer	Participant / Student
User Approval & Account Status	Full Control	View Profiles	Own Profile Only
Course & Lesson Creation	Full Control	Full Control	View Enrolled Only
AI Quiz Generation & Publishing	Full Access	Full Control	Take Quiz Only
Coding Assessment Setup & Testcases	Full Access	Full Control	Attempt & Submit
Live Proctoring Dashboard Monitoring	Global View	Own Course Live Monitor	No Access
Schedule Technical WebRTC Interview	Full Access	Schedule & Host	Join Scheduled Only
Submit Interview Evaluation & Feedback	View Reports	Evaluate & Grade	View Final Result
System Audit Logs & Global Analytics	Full Control	Course Metrics Only	Personal Progress

4. End-to-End System Workflows

Below are the 7 detailed operational workflows governing the platform. Each workflow highlights the sequential phases, component interactions, triggers, data changes, and security validations.

Workflow 1: User Registration, Onboarding & RBAC Governance

Workflow Phase	Detailed Operational Mechanism & Technical Execution
Phase 1: User Self-Registration	Candidate or Trainer submits registration form (Name, Email, Password, Role). System hashes password using bcrypt (salt 10) and stores record in `users` table with status `PENDING`.
Phase 2: Admin Security Verification	Administrator logs into Admin Dashboard, reviews pending requests, and approves/rejects accounts. Status transitions to `APPROVED`.
Phase 3: JWT Session Generation	User logs in with credentials. Backend validates bcrypt hash, issues signed JSON Web Token (JWT) with user ID, role, and expiration (24h), stored securely on client local storage/cookies.
Phase 4: Dynamic UI Route Protection	React Router checks token claims on client-side navigation. Express JWT middleware validates authorization headers on every API call.

Workflow 2: Course & Content Lifecycle Management

Workflow Phase	Detailed Operational Mechanism & Technical Execution
Phase 1: Course Structure Definition	Trainer inputs course details (Title, Description, Category, Level, Status). Backend creates entry in `courses` table with status `DRAFT`.
Phase 2: Lesson & Module Upload	Trainer creates modules and uploads content (PDFs, PPTs, Videos, external links). Multi-part upload processes files through Cloudinary / local storage.
Phase 3: Document AI Pre-Processing	Uploaded study materials (PDF/DOCX) are asynchronously enqueued to the AI Microservice for document chunking and vector store indexing.
Phase 4: Course Publishing & Student Enrollment	Trainer sets status to `PUBLISHED`. Candidates view available courses, submit enrollment requests, and access study materials upon approval.

Workflow 3: AI-Powered Assessment Generation (RAG Workflow)

Workflow Phase	Detailed Operational Mechanism & Technical Execution
Phase 1: Material Ingestion & Chunking	AI microservice parses lesson documents (`PyMuPDF`, `python-docx`). Text is chunked into 1,000 character overlapping segments.
Phase 2: FAISS Vector Embedding	HuggingFace `sentence-transformers` generates 768-dimensional embeddings for each chunk, storing them in an in-memory FAISS vector index.
Phase 3: LangChain Prompt & LLM Query	Trainer requests 10 MCQs on 'Database Normalization'. LangChain retrieves relevant chunks from FAISS and queries Google Gemini / OpenAI with structured schema constraints.
Phase 4: Question Verification & Quiz Publishing	AI returns JSON payload with Questions, Options, Correct Answer, Explanation, and Bloom's Level. Trainer reviews, edits, and publishes the quiz to the `quizzes` table.

Workflow 4: Automated Proctoring & Examination Security

Workflow Phase	Detailed Operational Mechanism & Technical Execution
Phase 1: Pre-Exam Readiness Check	Candidate initiates exam. Pre-exam screen verifies webcam access, microphone permissions, browser compatibility, and network speed.
Phase 2: Proctoring Telemetry Activation	System forces browser Fullscreen mode and attaches DOM event listeners for window blur, tab switch, right-click, copy/paste, and devtools keyboard shortcuts.
Phase 3: AI Alert Stream & Telemetry Logging	If candidate exits fullscreen or switches tab, a violation log is instantly recorded (`proctoring_logs`) and transmitted over Socket.IO to the Trainer Monitoring Dashboard.
Phase 4: Automated Exam Termination	If critical violation threshold (e.g., >3 tab switches) is exceeded, system automatically terminates session, submits current answers, and flags attempt for review.

Workflow 5: Sandbox Code Execution & Automated Evaluation

Workflow Phase	Detailed Operational Mechanism & Technical Execution
Phase 1: Coding Environment Setup	Candidate opens coding assessment. Monaco Editor loads problem statement, starter code, language selector (Python, JS, C++, Java), and testcases.
Phase 2: Client Code Dispatch	Candidate clicks 'Run Code' or 'Submit'. Code string, language ID, and test input are sent to `/api/coding-assessment/submit`.
Phase 3: Containerized Docker Execution	Backend delegates request to `dockerode` or Judge0 API. An isolated ephemeral Docker container starts with restricted CPU (0.5 cores), RAM (128MB), and network disabled.
Phase 4: Output Capture & Score Calculation	Container executes code against hidden test cases within a 2-second timeout window. Results (PASSED, FAILED, TLE, CE, RE) are compiled, calculated into a total score, and stored in `coding_submissions`.

Workflow 6: Native WebRTC Live Interview & Mobile Dual-Camera Pairing

Workflow Phase	Detailed Operational Mechanism & Technical Execution
Phase 1: Interview Scheduling & QR Generation	Interviewer schedules interview. System creates record in `interviews` table and generates secure room JWT token + mobile QR code.
Phase 2: Peer-to-Peer WebRTC Video Signalling	Interviewer and Candidate enter room `/interview/:id/room`. Socket.IO signalling server exchanges SDP Offers, Answers, and ICE Candidates. Peer-to-peer audio/video stream is established.
Phase 3: Secondary Mobile Camera Sync	Candidate scans QR code on smartphone. Mobile browser opens `/mobile-join/:token`, establishes a secondary WebRTC stream to interviewer, providing a 360° side-profile room monitor.
Phase 4: Synchronized Monaco Editor & Recording	Both parties collaborate live in a synchronized Monaco Code Editor via WebSockets. Client MediaRecorder captures streams, sending binary chunks to backend `/api/recordings` for archival.
Phase 5: Structured Scorecard & Final Decision	Interviewer completes 1-10 scorecard rating, feedback notes, and final decision (SELECTED, REJECTED, ON_HOLD) in `interview_feedback` and `interview_results` tables.

Workflow 7: Live Monitoring, Scorecards & Report Export

Workflow Phase	Detailed Operational Mechanism & Technical Execution
Phase 1: Real-Time Telemetry Aggregation	Trainer dashboard subscribes to Socket.IO live rooms. Live status feeds update active candidates, quiz progress, and security alert counters in real-time.
Phase 2: Comprehensive Candidate Analytics	System aggregates exam scores, proctoring violation scores, code efficiency, and interview ratings into a unified candidate performance profile.
Phase 3: Automated Report Generation	Using `exceljs` and server-side PDF renderer, Trainer exports official performance certificates, detailed evaluation transcripts, and compliance audit reports.

5. Database Schema & Data Dictionary

The relational data model comprises 35+ structured tables managed via Sequelize ORM with foreign key integrity, automated timestamp tracking, and optimized indexes.

Table Group	Core Tables	Primary Keys & Foreign Keys	Key Attributes & Enum Values
Identity & Auth	<code>`users`</code> <code>`trainer_profiles`</code> <code>`participant_profiles`</code>	PK: <code>`id`</code> FK: <code>`user_id` -> `users.id`</code>	<code>`role`</code> (ADMIN, TRAINER, PARTICIPANT) <code>`status`</code> (PENDING, APPROVED)
Courses & Content	<code>`courses`</code> <code>`lessons`</code> <code>`enrollments`</code> <code>`notes`</code>	FK: <code>`trainer_id` -> `users.id`</code> FK: <code>`course_id` -> `courses.id`</code>	<code>`status`</code> (DRAFT, PUBLISHED, ARCHIVED) <code>`material_type`</code> (PDF, VIDEO, PPT)
AI Quiz Engine	<code>`quizzes`</code> <code>`quiz_questions`</code> <code>`quiz_attempts`</code> <code>`quiz_answers`</code>	FK: <code>`quiz_id` -> `quizzes.id`</code> FK: <code>`user_id` -> `users.id`</code>	<code>`difficulty`</code> (EASY, MEDIUM, HARD) <code>`question_type`</code> (MCQ, SHORT_ANSWER)
Coding Sandbox	<code>`coding_assessments`</code> <code>`coding_questions`</code> <code>`coding_submissions`</code>	FK: <code>`assessment_id`</code> FK: <code>`question_id`</code>	<code>`status`</code> (PENDING, PASSED, FAILED, TLE, CE, RE) <code>`language`</code> (python, js, cpp, java)

Proctoring Logs	`exam_sessions` `proctor_logs` `proctor_violations`	FK: `session_id` -> `exam_sessions.id`	`violation_type` (TAB_SWITCH, FULLSCREEN_EXIT, COPY_PASTE) `severity` (INFO to CRITICAL)
WebRTC Interview	`interviews` `interview_sessions` `interview_devices` `interview_feedback`	FK: `interviewer_id` FK: `candidate_id` FK: `interview_id`	`status` (SCHEDULED, IN_PROGRESS, COMPLETED) `result` (SELECTED, REJECTED, ON_HOLD)

6. Security Architecture & Deployment Topologies

Enterprise Security & Isolation

- **Zero Untrusted Code Risk:** All student code runs inside non-root Docker containers with disabled networking, strict read-only filesystems, and cgroup memory limits.
- **Native WebRTC Signalling:** Peer-to-peer encrypted media streams (DTLS-SRTP). No third-party servers view or store video or audio streams.
- **Token Protection & RBAC:** Cryptographically signed JWT tokens with short lifetimes, HTTP-only cookies, and express-rate-limit brute force prevention.
- **SQL Injection & XSS Defenses:** Sequelize ORM parametrized queries, express-mongo-sanitize, Helmet security headers, and CORS whitelist policies.

Production Deployment Topology

The platform is container-ready and deploys via Docker Compose or Kubernetes clusters. A Cloudflare or Nginx reverse proxy handles SSL termination and WebSocket upgrading.

RECOMMENDED PRODUCTION DEPLOYMENT SETUP:

- **Load Balancer:** Nginx / Cloudflare (SSL Termination, WebSocket WSS routing)
- **App Nodes:** 2+ Node.js Instances (PM2 or Kubernetes Pods) with Redis Socket.IO Adapter
- **AI Microservice:** FastAPI Uvicorn workers with Gunicorn process manager
- **Code Execution:** Dedicated Docker Host running Docker daemon socket daemon
- **Database Tier:** Managed MySQL 8.0 Primary-Replica cluster + Redis Cluster

Prepared By: WAVEINIT Lead Engineering Team

Date: August 13, 2026

Approved By: Client Technical Review Board

Signature: _____